

Práctica 2

Límite para la entrega: 6 de noviembre, a las 23:59

Ordenación por inserción y ordenación por fusión**Ordenación por inserción**

Algoritmo de ordenación por inserción:

```
procedimiento ordenacionInsercionAux (var v[1..n], izq, der)
  para i := izq+1 hasta der hacer
    x := v[i] ;
    j := i-1 ;
    mientras j >= izq y v[j] > x hacer
      v[j+1] := v[j] ;
      j := j-1
    fin mientras ;
    v[j+1] := x
  fin para
fin procedimiento

procedimiento ordenacionInsercion (var v[1..n])
  ordenacionInsercionAux(v, 1, n)
fin procedimiento
```

Se pide:

1. Implemente el algoritmo de ordenación por inserción.¹

```
void ordenacionInsercion(int v[], int tam);
```

2. Valide el correcto funcionamiento de la implementación (figura 1).
3. Determine los tiempos de ejecución para distintos tamaños del vector y para tres diferentes situaciones iniciales: (a) el vector ya está ordenado en orden ascendente, (b) el vector ya está ordenado en orden descendente, y (c) el vector está inicialmente desordenado. (véase la figura 2).
4. Calcule empíricamente la complejidad del algoritmo para cada una de las diferentes situaciones iniciales del vector (figura 3).

¹**Nota sobre indexación:** El pseudocódigo está en base 1 (1..tam), mientras que en C se debe implementar en base 0 (0 a tam-1). Asegúrese de adaptar los límites para evitar desbordamientos de memoria.

```

> ./test
Ordenacion por insercion con inicializacion aleatoria
-1 9 5 -2 3 -8 -2 6 4 1
ordenando...
-8 -2 -2 -1 1 3 4 5 6 9      esta ordenado

Ordenacion por insercion con inicializacion descendente
17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1
ordenando...
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17      esta ordenado

```

Figura 1: Test

```

void inicializar_semilla() {
    srand(time(NULL));
}

void aleatorio(int v [], int n) { /* se generan números pseudoaleatorios entre -n y +n */
    int i, m=2*n+1;
    for (i=0; i < n; i++)
        v[i] = (rand() % m) - n;
}

void ascendente(int v [], int n) {
    int i;
    for (i=0; i < n; i++)
        v[i] = i;
}

```

Figura 2: Inicialización aleatoria y ascendente

```

Ordenacion por insercion con inicializacion aleatoria

```

	n	t(n)	t(n)/n ^{1.8}	t(n)/n ²	t(n)/n ^{2.2}
(*)	1000	371.030	0.00147710	0.00037103	0.00009320
	2000	1492.000	0.00170575	0.00037300	0.00008156
	4000	5839.000	0.00191704	0.00036494	0.00006947
	8000	23295.000	0.00219635	0.00036398	0.00006032
	16000	93651.000	0.00253569	0.00036582	0.00005278
	32000	371728.000	0.00289038	0.00036302	0.00004559

Figura 3: Parte de la posible salida por pantalla de la ejecución del programa principal

Ordenación por fusión

Algoritmo de ordenación por fusión (*mergesort*) con un umbral para detectar vectores pequeños:

```
procedimiento ordenacionFusionHibrida(var v[1..n], umbral)
    crearVector aux[1..n]
    ordenacionFusionHibridaAux(v, 1, n, aux, umbral)
fin procedimiento

procedimiento ordenacionFusionHibridaAux(var v[1..n], izq, der, var aux[1..n], umbral)
    si izq < der entonces
        si (der-izq+1) <= umbral entonces
            ordenacionInsercionAux(v, izq, der)
        sino
            medio := izq + (der - izq) div 2 ;
            ordenacionFusionHibridaAux(v, izq, medio, aux, umbral) ;
            ordenacionFusionHibridaAux(v, medio + 1, der, aux, umbral) ;
            fusionar(v, izq, medio, der, aux)
        fin si
    fin si
fin procedimiento

procedimiento fusionar(var v[1..n], izq, medio, der, var aux[1..n])
    i:= izq ; j:= medio + 1 ; k:= izq
    mientras i <= medio y j <= der hacer
        si v[i] <= v[j] entonces
            aux[k]:= v[i] ; k:= k+1 ; i:= i+1
        sino
            aux[k]:= v[j] ; k:= k+1 ; j:= j+1
        fin si
    fin mientras
    mientras i <= medio hacer
        aux[k]:= v[i] ; k:= k+1 ; i:= i+1
    fin mientras
    mientras j <= der hacer
        aux[k]:= v[j] ; k:= k+1 ; j:= j+1
    fin mientras
    para i:= izq hasta der paso +1 hacer
        v[i]:=aux[i]
    fin para
fin procedimiento
```

Se pide:

1. Implemente el algoritmo de ordenación por fusión:

```
void ordenacionFusionHibrida(int v[], int tam, int umbral);
```

2. Valide el correcto funcionamiento de la implementación.
3. Ejecute el algoritmo con vectores de distinto tamaño y en distintas situaciones iniciales (vector ordenado ascendente o descendientemente, o desordenado), y con distintos valores de umbral: 1 (no se realiza la llamada a la ordenación por inserción), 20 y 200.
4. Compare entre si los tiempos obtenidos para cada umbral usado. En función de la situación inicial del vector, ¿con qué umbral se obtienen los mejores tiempos? ¿por qué?
5. Calcule empíricamente la complejidad del algoritmo para cada una de las diferentes situaciones iniciales del vector y cada uno de los umbrales, es decir, 9 tablas (figura 4).

Ordenacion por fusion con umbral 20 e inicializacion descendente					
	n	t(n)	t(n)/n	t(n)/(n*ln(n))	t(n)/(n*ln ² (n))
(*)	1000	34.190	0.03419000	0.00494951	0.00071651
(*)	2000	74.642	0.03732100	0.00491007	0.00064599
(*)	4000	161.932	0.04048300	0.00488097	0.00058849
(*)	8000	349.042	0.04363025	0.00485471	0.00054018
(*)	16000	747.523	0.04672019	0.00482629	0.00049857
	32000	1633.000	0.05103125	0.00491939	0.00047423
	64000	3400.000	0.05312500	0.00480046	0.00043378
	128000	7202.000	0.05626562	0.00478458	0.00040686
	256000	15160.000	0.05921875	0.00475541	0.00038187
	512000	32029.000	0.06255664	0.00475858	0.00036198

Figura 4: Ordenación por fusión con umbral 20 sobre vectores inicializados descendientemente

Entregue los ficheros con el código C y el fichero .txt con el informe por medio de la tarea *Entrega Práctica 2* en el campus virtual. Se recuerda que el límite para completar la tarea es el viernes 6 de noviembre a las 23:59, y una vez subidos los archivos no se podrán cambiar. **Todos los compañeros que forman un equipo tienen que entregar el trabajo.**